Database Forensics

How Data Solves Crimes in Large Organisations

A 14-slide guide to investigating data at scale with SQL and audit logs

EVODIA UNJUEH AKU
unjuehevo@gmail.com

Day 5 of 30 | June 2026

DATABASE FORENSICS = DIGITAL EVIDENCE

TRANSACTION LOG - PART TWO

CONVERSION SYSTEMS

BINARY	OCTAL	DECIMAL	HEXA
Base: 2	Base: 8	Base: 10	Base: 16
Range: 0-1	Range: 0-7	Range: 0-9	Range: 0-15
Digits: 0,1	Digits: 0,1,2,3,4,5,6 AND 7	Digits: 0,1,2,3,4,5,6,7,8 AND 9	Digits: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E AND F

A. BINARY CONVERSIONS ON(1) AND OFF (0)

WHOLE NUMBERS

1. DIVISION RULE: DIVIDE BY 2 Repeatedly noting the Remainder always(0 or 1) 0 for even and 1 for odd.

- ❑ And read the remainder from **BOTTOM UP** and that is the equivalent
- ❑ Very efficient method can divide small numbers of 5 or as massive as 5,000,000

NUMBER 0: EXAMPLE 0/2 = 0

In computer science, **a single 0 isn't always just a 0.** computer stores data in fixed sizes called bytes (8 bits).
RULE: If a test or a program asks for the binary 0, its is ***never blank***. if they ask for a standard 8-bit byte representation, 0 is written as eight zeros **00000000**

2. SUBTRACTION RULE: To convert using this method, you compare your target number against the standard binary positional values(POWERS OF TWO)

- ❑ **If the number is greater than or equal to the positional value: put a 1, subtract the value, and move to the next column with the remainder.**
- ❑ **If the number is less than the positional value: put a 0, and move to the next column carrying the same number.**

SUBTRACTION RULE (TWO RULES)

1. Put 1 if:

Your Number >= Column Value

- ❑ Then subtract:

Your Number - Column value =

2. Put 0 if:

Your Number < Column Value

- ❑ Do not subtract. Just carry the exact same number over the next column

Binary Place Values

Position	Value
Ones	1
Tens	10
Hundreds	100
Thousands	1000

In binary, place values double each time

Position	Value
2^0	1
2^1	2
2^2	4
2^3	8
2^4	16
2^5	32
2^6	64
2^7	128

Switch	Binary
ON	1
OFF	0

Let's convert 13 to binary. write your answer also in 8 - byte

DIVISION METHOD

SUBTRACTION METHOD

2	13	RM		13	8	4	2	1	
2	6	1			5				1
2	3	0				1			1
2	1	1							0
		1					1		1
		1101							1101

00001101

00001101

Let's convert 16 to binary. write your answer also in 8 - byte

DIVISION METHOD

SUBTRACTION METHOD

2	16	RM			16	16	8	4	2	1	
2	8	0				0					1
2	4	0					0				0
2	2	0						0			0
2	1	0							0		0
		1								0	0
		1000 0									1000 0

00010000

00010000

BINARY FRACTIONS (DECIMAL POINTS)

Just like whole numbers, fraaction numbers use positional place values. When we move the right of the radix point(the decimal point). the column values **cut in half** instead of doubling.

The Fractional Place Value Table

Power	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}
Decimal Value	8	4	2	1	.	0.5	0.25	0.125	0.0625
Fraction					.	1/2	1/4	1/8	1/16

When presenting a combined number like 13.75, that the radix point (decimal point) acts like a mirror anchor.
Moving Left from the point: Values double (1 to 2 to 4 to 8).
Moving Right from the point: Values cut in half (0.5 to 0.25 to 0.125 to 0.0625).

FRACTIONAL SUBTRACTION METHOD

Best for: simple, clean decimal (like 0.75, 0.625, 0.5)

This works exactly like your whole-number subtraction rule, but you use the fractional column value

Example 13.75

WHOLE NUMBER FRACTION

8	4	2	1		1/2	1/4	1/8	1/16	

0.75 - 0.5 = 0.25 KEEP 1
0.25-0.25 = 0 KEEP 1
0 STOP

.11

1101.11

13
13-8 = 5 KEEP 1
5-4= 1 KEEP 1
1 KEEP 0
1-1=0 KEEP 1
0 STOP
1101

METHOD 2: The Continuous Multiplication Method

Best for: Complex decimals (like: 0.6) or numbers that don't easily map to fractions

Instead of dividing by 2 like we do for whole number WE MULTIPLY THE DECIMAL PART BY 2 Repeatedly

RULE:

- If the result is greater than or equal to 1, your binary digit is 1. You drop the 1 from the front and keep multiplying the remaining decimal.
- If the result is less than 1, your binary digit is 0. You keep multiplying.
- Read the answer from **TOP TO BOTTOM** (the opposite of the division rule).

Example: Convert 0.625 to Binary

0.625 X 2 = 1.25 KEEP 1
0.25 X 2 = 0.5 KEEP 0
0.5 X 2 =1 KEEP 1 STOP
0.00

. 101

16.625 = 10000.101

CRITICAL INSIGHT

Some clean decimal numbers can never be perfectly stored in binary.

Take the number 0.1 in decimal if you try to convert it using the multiplication method

.0001100011......

Once it gets to 1.2 the loop repeats infinitely

0.1 X 2 = 0.2 KEEP 0
0.2 X 2 = 0.4 KEEP 0
0.4 X 2 = 0.8 KEEP 0
0.8 X 2 = 1.6 KEEP 1
0.6 X 2 = 1.2 KEEP 1
0.2 X 2= 0.4 KEEP 0

It's going to keep looping.

In binary 0.1 becomes a repeating decimal: 0.0001100110011.....

Because computers don't have infinite storage bits, they have to round it off. This is called a floating-point rounding error, and in forensic systems auditing, it is exactly how tiny fractions of financial data or timestamp metrics can mysteriously drift over time!. This is exactly why database engineers prefer storing financial currency as whole integers (e.g., storing $10.50 as 1050 cents) to avoid mysterious data drift during audits.

RULE: If a decimal doesn't hit a clean .000 after 4 to 8 multiplications. STOP and write an ellipsis(...) or round it base on the bit deptg requirement .

Whole numbers: If a whole number is ODD. its 8-bIt representation must end in a 1. If it is EVEN, it must end in a 0
Decimal: If a decimal ends precisely in 0.5 (with no other remaining numbers), its binary fraction must end right after the slot (e.g ....1) If it ends in something like 0.25 or 0.75, it expands further.

Quick Conversion Examples

Decimal	Binary
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010

Bit

A single binary digit.

Examples:

0
1

Byte

8 bits together.

Example:

11001010

1 Byte = 8 Bits

Nibble

4 bits.

Example:

1010

Why Are Bytes Important?
Computers measure storage using bytes.
Examples:
1 Byte = 8 bits
1 Kilobyte (KB) ≈ 1,024 bytes
1 Megabyte (MB) ≈ 1,024 KB
1 Gigabyte (GB) ≈ 1,024 MB

5. What Can One Byte Store?

One byte can represent:

A letter
A number
A symbol

For example:

Character	Binary
A	01000001
B	01000010
C	01000011

The computer stores letters as binary numbers.

REVERSE THE ENGINE: BINARY DECIMALS TO BASE-10

The Golden Rule: "Just Add the ON Switches"
Every digit in a binary number sits in a specific "slot," and each slot has a fixed numerical value.

Switch	Binary
ON	1
OFF	0

If a slot has a 1 (ON), that value is active. You drop it down to add it.
If a slot has a 0 (OFF), that value is dead. You completely ignore it.

Let's convert the binary byte 00001101 back to a normal whole number.

Step 1: Draw Your 8-Bit Grid
Write out the standard 8 positional values from left to right, starting from 128 all the way down to 1.

128	64	32	16	8	4	2	1

Step 2: Slot the Binary Number In

Place your binary digits directly underneath those values, making sure the right side aligns perfectly.

00001101

128	64	32	16	8	4	2	1
0	0	0	0	1	1	0	1

Step 3: Collect the "ON" Values

Look across the table. Which columns have a 1 under them?

Step 4: Add Them Together

Take those active numbers and add them up like standard math: 1 + 4 + 8 = 13

FRACTIONS

Now, Let's Do a Fractional Decimal (Slowly)
What if the number has a point in it? Like 1101.11?
The rule stays exactly the same, but we use the "mirror" columns to the right of the decimal point where values cut in half (0.5, 0.25, etc.).
Step 1: Draw the Extended Grid Around the Point
Step 2: Slot the Number In

8	4	2	1		1/2	1/4
1	1	0	1		1	1

8 + 4 + 0 + 1 | 0.5 + 0.25
13.75

Step 3: Collect and Add Separately

1. Left Side (Whole Numbers): The columns with a 1
2.Right Side (Decimal Fractions):The columns with a 1

Step 4: Glue Them Back Together
Combine your whole number and your fraction around the decimal point: 13.75